

C89 Frontend Internals — How C Becomes Z80

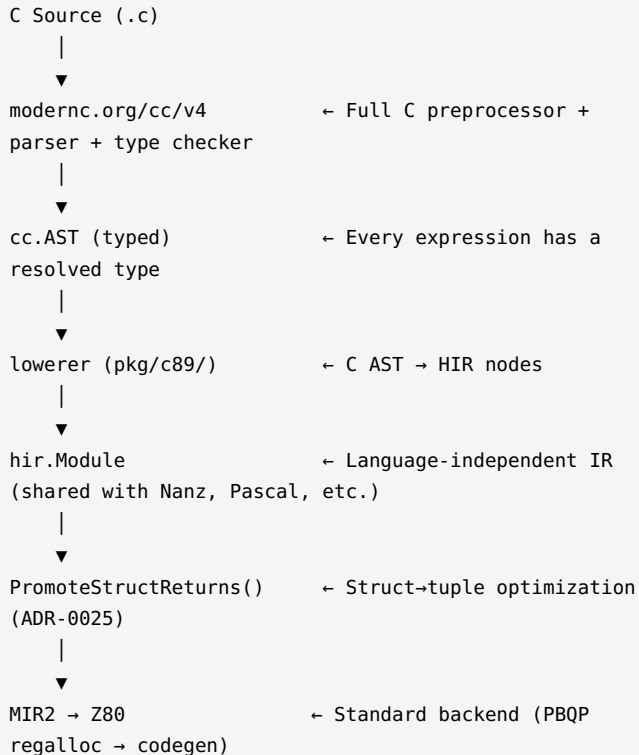
1. Pipeline Overview
 2. Parsing: moderncc.org/cc/v4
 3. Type Mapping: C Types → MIR2 Types
 4. Lowering: C Constructs → HIR
 5. Struct Handling
 6. Struct-Return Promotion (ADR-0025)
 7. Assert System
 8. Comparison with SDCC
- Appendix A: Source File Map
- Appendix B: Test Coverage

C89 Frontend Internals

— How C Becomes Z80

From `typedef struct` to LD A, (HL) in 7 stages.

1. Pipeline Overview



Entry point: `c89.CompileWithOpts(src, name, opts) → (*hir.Module, error)`

Key files: - `pkg/cparse/` — C parser (modernc.org/cc/v4 wrapper) - `pkg/c89/c89.go` — Main API, Z80 ABI, assert parsing - `pkg/c89/lower.go` — 1900+ LOC lowering logic - `pkg/c89/promote.go` — Struct-return promotion - `pkg/c89/libc.go` — Embedded minimal libc headers

2. Parsing: modernc.org/cc/v4

We don't write our own C parser. We use **modernc.org/cc/v4** — a production-quality C compiler frontend in pure Go. It gives us:

- Full C preprocessor (`#include`, `#define`, `#ifdef`)
- Complete C89/C99 parser
- Type system with implicit conversions and integer promotions
- AST with **fully resolved types** (not just syntax trees)

Z80 ABI Configuration

```
func z80ABI() *cc.ABI {  
    return &cc.ABI{
```

```

        ByteOrder: binary.LittleEndian,
        SignedChar: true,
        Types: map[cc.Kind]cc.AbiType{
            cc.Char: {Size: 1, Align: 1},
            cc.Int:  {Size: 2, Align: 1},    // 16-bit
int
        cc.Ptr:   {Size: 2, Align: 1},    // 16-bit
pointers
        cc.Long: {Size: 4, Align: 1},    // 32-bit
long
        },
    },
}

```

Predefined macros: `__Z80__ 1`, `__MINZ__ 1`,
`__SIZEOF_INT__ 2`, plus full `stdint.h`
 typedefs (`uint8_t`, `int16_t`, etc.) pre-
 injected.

3. Type Mapping: C Types → MIR2 Types

Z80 has 5 fundamental types. Everything maps to them:

C Type	MIR2 Type	Z80 Size	Notes
void	TyVoid	0	—
char, unsigned char	TyU8	1	signed char → TyI8
short, int (signed)	TyI16	2	Z80 native word
unsigned int	TyU16	2	—

C Type	MIR2 Type	Z80 Size	Notes
<code>_Bool</code>	<code>TyBool</code>	1	Normalized to 0/1
<code>T *</code> , function pointers	<code>TyPtr</code>	2	All pointers are 16-bit
<code>T[]</code>	<code>ArrayTy(elem, len)</code>	—	Preserves length
<code>struct S</code>	<code>TyPtr</code>	2	Pointer to struct data
<code>enum E</code>	<code>TyU8</code> or <code>TyU16</code>	—	Based on value range
<code>float</code> , <code>double</code>	<code>TyU16 (stub)</code>	2	Not truly supported

Key insight: Structs are lowered to pointer types because Z80 cannot pass structs in registers. Struct definitions are stored separately in `hir.Module.Structs[]`, and field access compiles to base-address + offset.

4. Lowering: C Constructs → HIR

4.1 Functions

```
uint8_t add(uint8_t a, uint8_t b) {
    return a + b;
}
```

Becomes:

```
fun @add(a: u8, b: u8) -> u8
  return ((a:u8) + (b:u8)):u8
```

Two phases: 1. **Signature:** Extract params, map types, map return type 2.

Body: Lower compound statement recursively

4.2 Control Flow

C Statement	HIR Equivalent
if (c) { ... } else { ... }	hir.IfStmt{Cond, Then, Else}
while (c) { ... }	hir.WhileStmt{Cond, Body}
for (init; c; post) { ... }	Desugars to init; while(c) { body; post }
switch/case	Desugars to if/else chain (no fall-through)
break / continue	hir.BreakStmt / hir.ContinueStmt
goto label / label:	hir.GotoStmt / hir.LabelStmt
return expr	hir.ReturnStmt{Val: expr}

Switch desugaring example:

```
switch (x) {
  case 1: doA(); break;
  case 2: doB(); break;
```

```

    default: doC();
}

```

→

```

if (x == 1) { doA() }
else if (x == 2) { doB() }
else { doC() }

```

4.3 Expressions

C Expression	HIR Node
<code>a + b</code>	<code>BinExpr{Op:"+", L, R, Ty}</code>
<code>a && b</code>	<code>CondExpr{a, b!=0, 0} (short-circuit)</code>
<code>a ? b : c</code>	<code>CondExpr{Cond, Then, Else}</code>
<code>(uint8_t)x</code>	<code>CastExpr{X, TyU8}</code>
<code>&x</code>	<code>AddrOfExpr{Sym: "x"}</code>
<code>*p</code>	<code>LoadExpr{Ptr, ElemTy}</code>
<code>arr[i]</code>	<code>IndexExpr{Base, Idx, ElemTy}</code>
<code>s.field</code>	<code>FieldExpr{X, Field, Offset}</code>
<code>p->field</code>	Same as <code>.field</code> (pointer unwrapped)
<code>f(args)</code>	<code>CallExpr{Fn, Args, Ty}</code>
<code>fp(args) (via ptr)</code>	<code>CallIndirectExpr{FnPtr, Args, Ty}</code>
<code>x++</code>	Side-effect: <code>tmp=x; x=x+1; result=tmp</code>

4.4 Side-Effect Buffering

C expressions can embed assignments (a = b = c, x++). The lowerer uses `exprResult` wrapper:

```
int x = y++;
```

→

```
__post_0 = y      // save old value
y = y + 1         // increment
int x = __post_0  // use old value
```

Pending side-effect statements are buffered in `fl.pendingStmts` and drained before the next statement.

4.5 Integer Narrowing

C promotes `u8` operands to `int` (16-bit). For Z80 efficiency, the lowerer **selectively undoes** this:

```
switch op {
case "-", "/", "%", "&", "|",
"^": // safe: result fits in u8
    result_type = u8
case "+", "*", "<<": // NOT safe:
    overflow
    result_type = u16
}
```


This preserves 8-bit ALU operations where correct.

5. Struct Handling

5.1 Declaration

Structs are lowered with embedded struct flattening:

```
struct Point { int x; int y; };  
struct Rect { struct Point origin; int width; };
```

→

```
Rect {  
  origin.x: i16,  
  origin.y: i16,  
  width: i16  
}
```

Flattening ensures byte offsets are correct without nested FieldExpr chains.

5.2 Unions

Unions are `mir2.StructTy{IsUnion: true}` — all fields overlap at offset 0, typed as the largest member.

5.3 Type Resolution

Three-way mapping handles all struct patterns: 1. structs[tag] — tag name → MIR struct 2. cStructs[cc.Type] — C type object → tag (handles typedefs) 3. Field-count matching — fallback for anonymous typedef struct { ... } Name

5.4 Static Locals

```
void counter(void) { static int count = 0; count++; }
```

→ count becomes global counter__count (mangled name).

6. Struct-Return Promotion (ADR-0025)

The **key optimization** for C-on-Z80 performance.

Problem

SDCC returns small structs via stack (~60T overhead). Z80 registers can hold up to 4 bytes.

Solution

Automatically promote struct-returning functions to tuple returns, enabling register allocation.

Shallow Scalar Struct (SSS)

A struct is SSS if: - All fields are scalar (u8, u16, i8, i16, bool) - No nested structs, no arrays - ≤ 4 bytes total, ≤ 4 fields

Four Phases

Phase 0: Out-Parameter Promotion

```
void divmod(uint8_t a, uint8_t b, DivResult *out) {  
    out->q = a / b;  
    out->r = a % b;  
}
```

Detection: last param is TyPtr, body only does `out->field = expr` (pure write-only).

→ Becomes:

```
fun @divmod(a: u8, b: u8) -> (u8, u8)  
    return (a / b, a % b)
```

Call sites rewritten: `divmod(a, b, &res);`
`use(res.q) → let (q, r) = divmod(a, b);`
`use(q)`

Phase 1: Struct-Return Promotion

Three return patterns detected: 1. Direct:
`return (DivResult){a/b, a%b}` 2. Indirect:
`DivResult res = {...}; return res;` 3.
Pointer-via-field:
`res.q = ...; res.r = ...; return &res;`

Phase 2: Signature Rewrite

`RetTy: TyPtr → RetTys: [TyU8, TyU8] +
tuple return statements.`

Phase 3: Call Site Rewrite

`DivResult d = divmod(17, 5); use(d.q); →
let (d_q, d_r) = divmod(17, 5); use(d_q);`

Phase 4: Dead Code Elimination

Remove orphaned `StructLitExpr VarDecls`.

Performance Impact

Case	SDCC	After Promotion
No addressable fields	~60T	0T
One field addressable	~60T	7T
Worst case	~60T	14T

Bugs Fixed (2026-03-24)

1. **AddrOfExpr not handled** — C89 emits `AddrOfExpr{Sym}` for `&var`, not `UnaryExpr{Op: "&"}`. Call site rewrite now handles both.
 2. **Void return not stripped** — C89 emits `return;` for void functions. After promotion to tuple, the void return executed first, making tuple return unreachable. Now filtered.
-

7. Assert System

Comment-based compile-time assertions, verified via MIR2 VM:

```
// assert fn(1, 2) == 42 via mir2
// assert fn(0xFF) == 0 via z80

// sandbox "stateful_tests"
// assert inc() == 1
// assert inc() == 2
// assert get() == 2
// end sandbox
```

- **Top-level asserts:** each gets fresh VM instance
- **Sandbox asserts:** share one VM instance (sequential, shared state)

- **via mir2**: MIR2 VM only; **via z80**: Z80 emulator only; **omitted**: both

Parsed by `parseAssertLine()` using regex, stored as `hir.Assert` in module.

8. Comparison with SDCC

Feature	SDCC	MinZ C89
Struct return	Stack-based (~60T)	Tuple promotion (0T)
Register allocation	Graph coloring	PBQP + WFC
Peephole patterns	~200	67 + MIR passes
Integer width	8/16/32-bit	8/16-bit (32 via library)
Optimization	Per-function	Whole-program
Cross-language	C only	C + Nanz + Pascal + 6 more
Output	Assembly	Assembly + binary
Float support	Software float	Not supported
Runtime	Full libc	Minimal (embedded)

Code Size (Book Examples, 2026-03-24)

LIR backend produces **22% smaller code** overall vs legacy PBQP on Nanz book examples. On complex C programs (enum/match/struct), savings reach 50-70%.

Appendix A: Source File Map

```
pkg/cparse/
├─ lexer.go      – Tokenizer (via modernc.org/cc)
├─ parser.go     – Parser wrapper
├─ type.go       – Type system (Kind, PointerType,
ArrayType, etc.)
└─ ast.go        – AST nodes

pkg/c89/
├─ c89.go        – Entry point, Z80 ABI, assert
parsing
├─ lower.go      – Main lowerer (1900+ LOC)
├─ promote.go    – Struct-return promotion (4
phases)
├─ libc.go       – Embedded libc headers
├─ objc_lower.go – Objective-C extension
├─ c89_test.go   – Unit tests + E2E
└─ promote_test.go – Promotion-specific tests
```

Appendix B: Test Coverage

Test	What It Verifies
TestCompile_VoidFunc	Basic function lowering
TestCompile_AddFunction	Parameter & return type mapping

Test	What It Verifies
TestCompile_IfElse	Control flow lowering
TestCompile_WhileLoop	Loop desugaring
TestCompile_CommentAsserts	Assert directive parsing
TestPromote_DivmodStruct	Full promotion pipeline
TestPromote_OutParam	Out-param → tuple
TestPromote_PtrReturn	Pointer-return → tuple
TestPromote_SSS_Detection	SSS criteria validation

C89 Frontend: making SAP consultants' C code run on a ZX Spectrum since 2026.